

MPN

GUIDED BY VISION

MPN

GUIDED BY VISION

Indholdsfortegnelse:

Forklaring af system: ...4

Teknisk forklaring af system: ...6

Miljø og forudsætninger: ...9

Fejl forklaring: ...11

MPN

GUIDED BY VISION

 Dokumentation af UI-applikation

Forklaring af system

Ikke teknisk beskrivelse:

Dette system er en automatiseret måde at bygge og pakke et lille UI program på. Det bruger en Dockerfile, som er en slags opskrift, der fortæller computeren, hvordan den skal samle programmet og dets nødvendige dele.

Dette system gør det nemt for både udviklere at bygge, pakke og installere et program uden at skulle dykke ned i komplekse tekniske detaljer. Det sikrer, at programmet fungerer korrekt og er let at dele og installere på forskellige 'barebone'-linux systemer.

Denne løsning er designet til at gøre det nemt at udvikle og teste en lille, selvstændig brugerflade direkte i din webbrowser. Det betyder, at du kan arbejde med og afprøve din grafiske applikation lokalt, uden at skulle installere den på en maskine.

Når du er tilfreds med, hvordan det hele ser ud og fungerer i din browser, kan du pakke det hele sammen til en komplet AppImage. Denne pakke indeholder alt, hvad der er nødvendigt for at køre applikationen på en hvilken som helst linux system, uden behov for yderligere installationer eller internetforbindelse.

MPN

GUIDED BY VISION

 Dokumentation af UI-applikation

Teknisk forklaring af system

Dockerfile for NetSurf Framebuffer AppImage (32-bit)

Formål

Dockerfilen bygger en selvstændig AppImage af NetSurf-webbrowseren til framebuffer (uden grafisk brugerflade), målrettet 32-bit Linux-systemer. Den samler alle nødvendige biblioteker og værktøjer, så browseren kan køre uden afhængigheder på målmaskinen.

1. Basebillede

Starter med et 32-bit Ubuntu 16.04-billede, som bruges til at bygge browseren og dens afhængigheder.

2. Installation af afhængigheder

Installerer alle nødvendige systemværktøjer og biblioteker:

- Kompilatorer (build-essential, bison, flex)
- Netværksbiblioteker (libcurl, libssl)
- Grafikbiblioteker (libpng, libjpeg)
- Værktøjer som imagemagick og cmake

3. Miljøvariabler

Sætter stier for, hvor biblioteker og værktøjer skal installeres og findes under bygningen.

4. Installation af CMake

Downloader og installerer en nyere version af CMake (v3.25.2), som kræves til nogle af komponenterne.

5. Hent og installer NetSurf-byggesystem og kildekode

Henter og pakker kildekode for:

- NetSurf-biblioteker (libcss, libdom, libhubbub osv.)
- NetSurf-browserens framebuffer-version

6. Byg og installer biblioteker

Bygger og installerer hver afhængighed ét ad gangen til den definerede installationssti.

7. Byg NetSurf-browseren til framebuffer

Kompilerer selve NetSurf-browseren uden støtte for f.eks. JavaScript eller JPEG, for at forenkle afhængighederne.

8. Opret AppDir-struktur

Forbereder mappestruktur som krævet af AppImage-formatet. Kopierer browserens binære fil og tilhørende data.

9. Opret ikon og desktop entry

Opretter en dummy-ikon med imagemagick. Tilføjer en .desktop-fil så AppImage vises som en applikation i skrivebordsmiljøer.

10. Kopiér og bundt alle nødvendige biblioteker

Finder og kopierer alle delte biblioteker, som NetSurf afhænger af. Løser også recursivt afhængigheder for bibliotekerne.

11. Patche biblioteker og binær

Sørger for, at NetSurf bruger de medfølgende biblioteker i AppDir ved at ændre RPATH. Bevarer korrekt linkning uden at ændre den dynamiske linker.

12. Opret startscript (AppRun)

Skriver et script, som starter NetSurf med korrekt miljø, inklusive brugerdefineret linker og ressource-sti.

13. Byg selve AppImage

Henter appimagetool, som bruges til at bygge AppImage. Kører værktøjet og pakker AppDir til én enkelt .AppImage fil.

14. Eksporter resultatet

Overfører den færdige AppImage fra builder til en minimal fase (scratch) for nem eksport.

Resultat

Efter at have bygget dette Docker-image, får man en selvstændig kørbare fil:

netsurf-fb-i386.AppImage

Den kan afvikles på ethvert kompatibelt 32-bit Linux-system uden behov for yderligere installation.

MPN

GUIDED BY VISION

 Dokumentation af UI-applikation

Miljø og forudsætninger

Dette projekt er udviklet og testet med henblik på at køre på et mål-system baseret på **Ubuntu 16.04 med en meget minimal kerne**, som anvendes på CP6000-maskinen. Målmiljøet stiller begrænsede systemressourcer og manglende moderne biblioteker til rådighed, hvilket stiller særlige krav til kompatibilitet og afhængighedsstyring.

Udviklingsmiljø

- **Host-OS:** Windows 10/11
- **Udviklingsmiljø:** WSL2 (Windows Subsystem for Linux) med Ubuntu
- **Docker-version:** 24.x (kørt via WSL)
- **Kompatibilitet:** Bygget til 32-bit systemer (`linux/386` platform)

Målmiljø (Target system)

- **Maskine:** CP6000
- **OS:** Ubuntu 16.04 (32-bit)
- **Kernel:** Minimal (mange moderne afhængigheder ikke til stede som standard)
- **Brug af AppImage:** Den kompilerede og patched browser pakkes som en selvstændig AppImage, som testes på en **virtuelt Ubuntu 16.04 miljø**, der simulerer målplatformens forhold.

Begrænsninger og krav

- Alle afhængigheder skal inkluderes statisk eller bundtes ved hjælp af `patchelf` og placeres lokalt.
- Målmiljøet understøtter ikke nyere versioner af `libssl`, `libcrypto` m.fl., hvilket kræver nøje versionering og manuel afhængighedshåndtering.
- Det er ikke muligt at køre moderne browsere som Firefox eller Chromium direkte, hvilket motiverer brugen af NetSurf i framebuffer-mode.

MPN

GUIDED BY VISION

 Dokumentation af UI-applikation

Fejl forklaring

Fejlhåndtering: Segmentation Fault ved kørsel af netsurf - fb

Problem:

Under kørsel af den kompilerede netsurf - fb binærfil i den færdige Docker container opstår følgende fejl:

```
java
Segmentation fault (core dumped)
```

Denne fejl indikerer, at programmet forsøger at tilgå en hukommelsesadresse, som det ikke har adgang til. Det er en klassisk lav-niveau fejl, der typisk opstår i C/C++ programmer ved f.eks. dereference af NULL pointere eller buffer overflows.

Sandsynlige Årsager

I dette projekt er der nogle oplagte potentielle kilder til problemet:

1. Manglende eller inkompatible afhængigheder:

Selvom shared libraries (*.so) kopieres manuelt til /build-out/libs, er det muligt at nogle nødvendige runtime-afhængigheder enten:

- Ikke bliver fundet korrekt pga. forkert LD_LIBRARY_PATH
- Er inkompatible versioner (f.eks. libssl.so.3 i stedet for libssl.so.1.1)

2. Forkert eller manglende interpreter (dynamic linker):

Hvis ld-linux.so.2 ikke er korrekt patchet eller ikke matcher glibc-versionen brugt under kompileringen, kan det føre til segmentation faults.

3. Forkert RPATH eller afhængighedsstruktur:

Hvis RPATH ikke er sat korrekt med patchelf, vil programmet ikke kunne finde sine biblioteker ved runtime, og det kan fejle ved første kald til dynamiske symboler.

4. Miljøvariabler:

Miljøvariabler som NETSURFRES eller LD_LIBRARY_PATH skal være sat korrekt, da NetSurf afhænger af at kunne finde ressourcer og biblioteker på runtime.

